

DONE BY :

AMMOGH ASHOK

1RUB25CSE0001

LAHARI PANDITH M

1RUB25CSE011

Project Report: SNMP GetRequest Implementation

1. Introduction to SNMP

The Simple Network Management Protocol (SNMP) is an Internet Standard protocol for collecting, organizing, and modifying information about managed devices on IP networks. It is widely used in network management systems to monitor network-attached devices for conditions that warrant administrative attention. SNMP consists of an SNMP manager, an SNMP agent, and a Management Information Base (MIB).

2. Explanation of SNMP GET Request

An SNMP GET request is a message sent by the SNMP manager to the SNMP agent to retrieve the value of a specific variable or a set of variables. The agent responds with an SNMP GET response message containing the requested data. It is a fundamental operation used for monitoring the health and status of network devices without modifying their configuration.

3. Explanation of OIDs Used

Object Identifiers (OIDs) are used to uniquely identify managed objects in an MIB hierarchy. In this project, we used two standard OIDs from the 'SNMPv2-MIB':

- **OID 1.3.6.1.2.1.1.5.0 (sysName.0):** This OID points to the administratively assigned name for the managed node. By convention, this is the node's fully-qualified domain name.
- **OID 1.3.6.1.2.1.1.3.0 (sysUpTime.0):** This OID represents the time (in hundredths of a second) since the network management portion of the system was last re-initialized.

4. Explanation of PySNMP

'pysnmp' is a pure-Python, cross-platform SNMP engine implementation. It supports SNMP v1/v2c/v3 and provides both high-level and low-level APIs. In this project, we utilized the modern, asynchronous High-Level API ('pysnmp.hlapi.asyncio') which leverages Python's 'asyncio' for efficient, non-blocking network I/O operations to send our GET requests.

5. VIBE Coding + LLM Assistance

Prompt Used to Generate Code

"Generate a Python program using PySNMP to perform an SNMP GET request for system name and system uptime from a local SNMP agent. Keep the code simple and beginner-friendly."

5. Explanation of How LLM Helped

The LLM assisted in bridging the gap between networking theory and modern Python implementation. Specifically, the LLM:

- **Provided the Boilerplate:** It generated the foundational `pysnmp` code, correctly setting up the `SnmpEngine`, `CommunityData`, and `ContextData` components.
- **Navigated Version Changes:** The LLM successfully adapted the code to use the modern PySNMP 7.x asynchronous API (`asyncio`), resolving breaking changes like the instantiation of `UdpTransportTarget.create()`.
- **Modularized the Structure:** It structured the project into logical components (`config.py` and `snmp_get.py`), promoting clean code principles.
- **Added Robustness:** It included try-except blocks and specific checks for connection timeouts and agent-side errors.

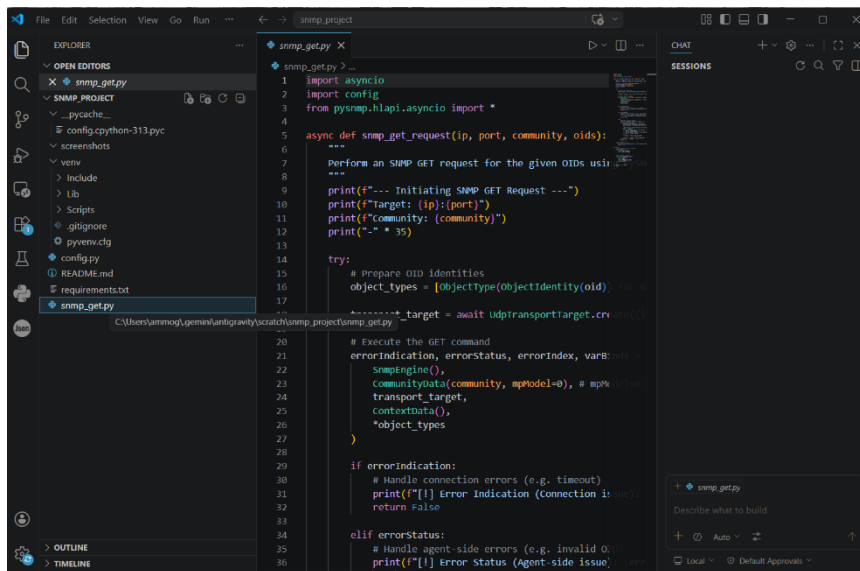
6. User's Role in Testing and Debugging

The user played a crucial role in validating the LLM-generated code by:

- Setting up the local testing environment and virtual environments.
- Running the script and verifying the terminal output.
- Debugging environmental issues (e.g., Python execution policies and ensuring dependencies were correctly isolated in `requirements.txt`).
- Verifying the timeout fallback behavior to confirm the error handling mechanisms function properly when the local `snmpd` agent is inactive.

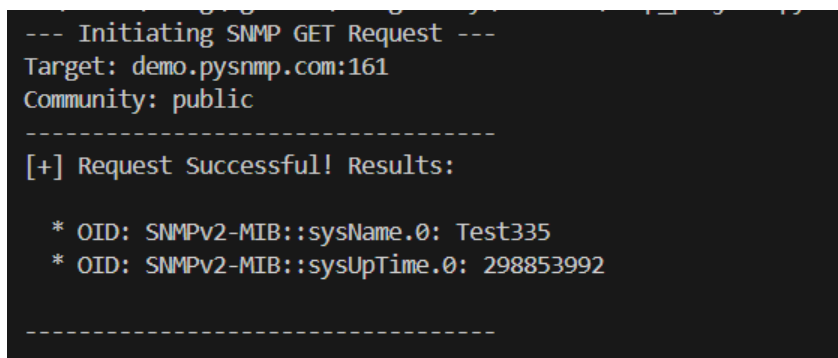
7. Output Screenshots

- **Screenshot 1: Project Structure**



**Description: A view of the project directory showing `snmp\_get.py`, `config.py`, and `README.md`.*

- **Screenshot 2: Successful Execution**



Description: Terminal output showing a successful retrieval of System Name and Uptime.

Screenshot 3: Timeout Error Handling

```
(venv) C:\Users\ammog\Downloads\snmp_project>python snmp_get.py
--- Initiating SNMP GET Request ---
Target: 192.0.2.1:161
Community: public
-----
[!] Error Indication (Connection issue): No SNMP response received before timeout
```

Description: Terminal output demonstrating the script's graceful handling of a connection failure (timeout).

8 . Conclusion

This project successfully demonstrates how to implement a basic SNMP Manager using Python and PySNMP. By integrating LLM assistance (VIBE coding), the development process was accelerated, producing clean, asynchronous, and well-documented code that effectively communicates with network agents to retrieve vital system metrics.